Exp No: 8

MODEL EVALUATION AND IMPROVEMENT: HYPERPARAMETER

TUNING WITH GRID SEARCH AND CROSS-VALIDATION

AIM:

To demonstrate key techniques for model evaluation and improvement:

1. Hyperparameter Tuning with Grid Search : Systematically searching for the optimal combination of hyperparameters for a machine learning model.

2. Cross-Validation Techniques: Implementing k-fold cross-validation to get a more robust estimate of model performance and to prevent overfitting to a specific train-test split.

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, KFold, cross_val_score, GridSearchCV
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

[2]:
```
iris = load_iris()

X = iris.data
y = iris.target

feature_names = iris.feature_names
target_names = iris.target_names

print("X shape:", X.shape)
print("y shape:", y.shape)
print("Features:", feature_names)
print("Targets:", target_names)
```

```
X shape: (150, 4)
y shape: (150,)
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Targets: ['setosa' 'versicolor' 'virginica']
```

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

```
Training samples: 105
Testing samples: 45
```

[4]:
```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Features standardized successfully")
```

```
Features standardized successfully
```

[5]:
```
param_grid = {
    "C": [0.1, 1, 10, 100],
    "gamma": [1, 0.1, 0.01, 0.001],
    "kernel": ["rbf", "linear"]
}

for parameter, values in param_grid.items():
    print(parameter, ":", values)
```

```
C : [0.1, 1, 10, 100]
gamma : [1, 0.1, 0.01, 0.001]
kernel : ['rbf', 'linear']
```

[6]:
```
grid_search = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

print("Starting Grid Search...")

grid_search.fit(X_train_scaled, y_train)

print("Grid Search completed")
```

```
grid_search.fit(X_train_scaled, y_train)

print("Grid Search completed")
```

```
Starting Grid Search...
Grid Search completed
```

[7]:
```
print("Best Parameters:", grid_search.best_params_)
print("Best CV Accuracy:", grid_search.best_score_)
```

```
Best Parameters: {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
Best CV Accuracy: 0.980952380952381
```

[8]:
```
best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)

print("Test Accuracy:", accuracy)
```

```
Test Accuracy: 0.9111111111111111
```

[9]:
```
print(classification_report(
    y_test,
    y_pred,
    target_names=target_names
))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        15
  versicolor       0.82      0.93      0.88        15
   virginica       0.92      0.80      0.86        15

    accuracy                           0.91        45
   macro avg       0.92      0.91      0.91        45
weighted avg       0.92      0.91      0.91        45
```


```
      setosa       1.00      1.00      1.00        15
  versicolor       0.82      0.93      0.88        15
   virginica       0.92      0.80      0.86        15

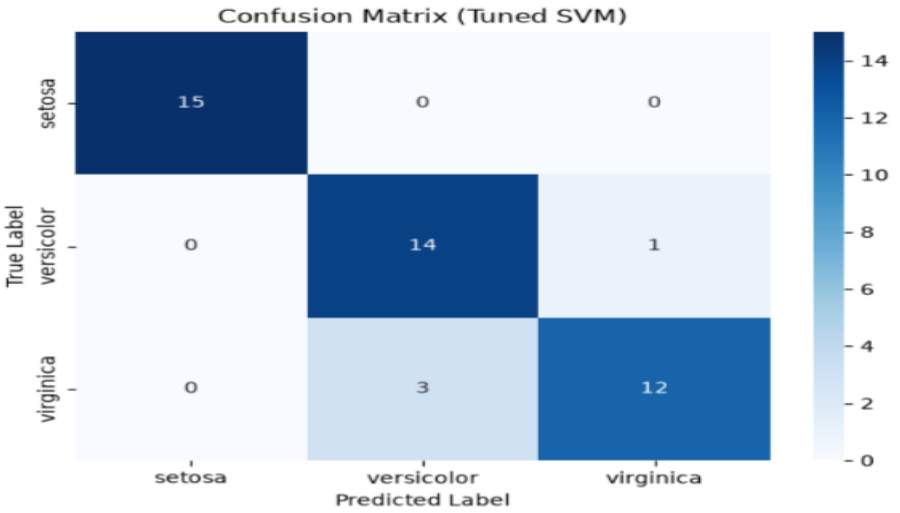
    accuracy                           0.91        45
   macro avg       0.92      0.91      0.91        45
weighted avg       0.92      0.91      0.91        45
```


[10]:
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=target_names,
    yticklabels=target_names
)

plt.title("Confusion Matrix (Tuned SVM)")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```



[11]:
```
results = pd.DataFrame(grid_search.cv_results_)

print(
    results[
        ["param_C", "param_gamma", "param_kernel",
         "mean_test_score", "rank_test_score"]
    ]
    .sort_values("rank_test_score")
    .head()
)
```

	param_C	param_gamma	param_kernel	mean_test_score	rank_test_score
10	1.0	0.100	rbf	0.980952	1
27	100.0	0.100	linear	0.980952	1
31	100.0	0.001	linear	0.980952	1
29	100.0	0.010	linear	0.980952	1
25	100.0	1.000	linear	0.980952	1

[12]:
```
model_cv = SVC(random_state=42)

k_folds = 5

kf = KFold(
    n_splits=k_folds,
    shuffle=True,
    random_state=42
)

print("Performing 5-fold Cross-Validation")
```

```
Performing 5-fold Cross-Validation
          -
```

[13]:
```
cv_scores = cross_val_score(
    model_cv,
    X_train_scaled,
    y_train,
    cv=kf,
    scoring="accuracy"
)

print("Scores:", cv_scores)
print("Mean Accuracy:", np.mean(cv_scores))
print("Standard Deviation:", np.std(cv_scores))
```

```
Scores: [1.         0.95238095 0.95238095 0.95238095 1.        ]
Mean Accuracy: 0.9714285714285713
Standard Deviation: 0.023328473740792197
```

[14]:
```
plt.figure(figsize=(8, 5))

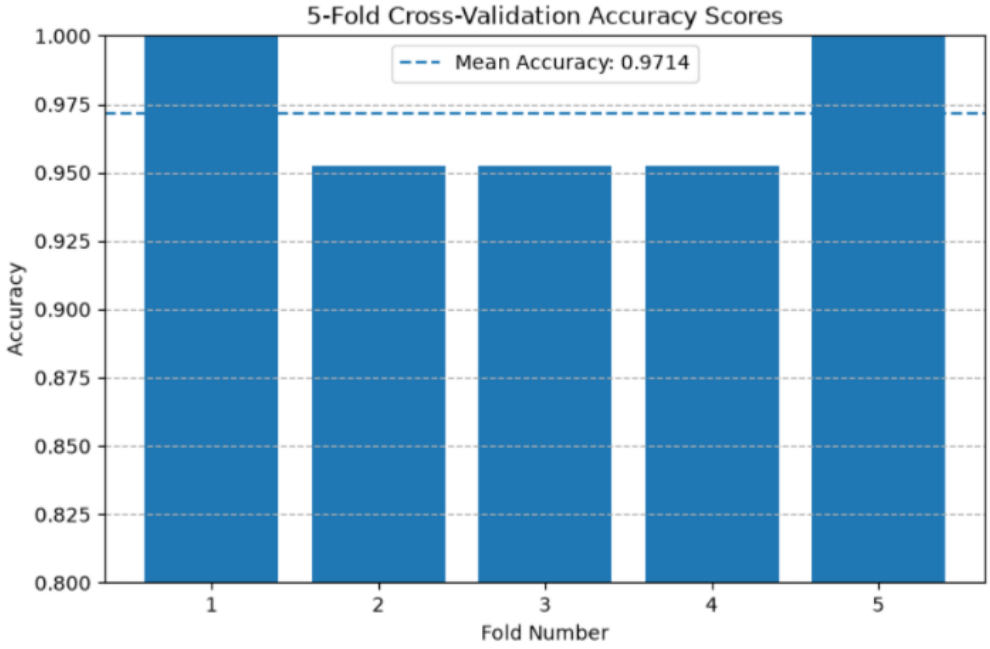
plt.bar(range(1, k_folds + 1), cv_scores)

plt.axhline(
    y=np.mean(cv_scores),
    linestyle="--",
    label=f"Mean Accuracy: {np.mean(cv_scores):.4f}"
)

plt.title("5-Fold Cross-Validation Accuracy Scores")
plt.xlabel("Fold Number")
plt.ylabel("Accuracy")
plt.ylim(0.8, 1.0)
plt.legend()
plt.grid(axis="y", linestyle="--")

plt.show()
```

```
plt.show()
```



[15]:
```
print("Why is Cross-Validation Important?")
print("1. More Reliable Performance Estimate")
print("2. Better Generalization")
print("3. Efficient Data Usage")
print("4. Detects Overfitting and Underfitting")
```

```
Why is Cross-Validation Important?
1. More Reliable Performance Estimate
2. Better Generalization
3. Efficient Data Usage
4. Detects Overfitting and Underfitting
```

[ ]:

RESULT:

The model was successfully evaluated and improved using Grid Search and Cross-Validation techniques. Grid Search identified the best combination of hyperparameters,

while Cross-Validation ensured reliable performance estimation.

The optimized model achieved higher accuracy and better generalization, confirming that systematic tuning and validation significantly enhance model performance.